

Инструменты разработчика

Н.А. Пономарев

25 сентября 2026 г.

СПбГУ

n.ponomarev@spbu.ru

Какие инструменты разработчика вы знаете?

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Зачем форматтер

```
data=[[1,2],[3,4],[5,6]]
for lst in data:
    s=0
    for n in lst:
        s+=n
    print(s)
```

```
data = [[1, 2], [3, 4], [5, 6]]
for lst in data:
    s = 0
    for n in lst:
        s += n
    print(s)
```

Зачем форматтер

```
data=[[1,2],[3,4],[5,6]]
for lst in data:
    s=0
    for n in lst:
        s+=n
    print(s)
```

```
data = [[1, 2], [3, 4], [5, 6]]
for lst in data:
    s = 0
    for n in lst:
        s += n
    print(s)
```

Смысл тот же, читать и сопровождать — второй вариант проще. Форматтер делает это автоматически

Зачем анализаторы

```
price = input("Цена: ")  
total = price + price * 0.15
```

Зачем анализаторы

```
price = input("Цена: ")  
total = price + price * 0.15
```

Кажется, всё по правилам. Но проверка типов заметит, что `price` — строка, и укажет на проблему **до запуска**

Зачем тесты

```
def is_sorted(xs):  
    for i in range(1, len(xs) - 1):  
        if xs[i - 1] > xs[i]:  
            return False  
    return True
```

Зачем тесты

```
def is_sorted(xs):  
    for i in range(1, len(xs) - 1):  
        if xs[i - 1] > xs[i]:  
            return False  
    return True  
  
>>> is_sorted([1, 2, 3, 0])  
True
```

Функция выглядит корректно, но на границе списка пропускает последнюю пару.
Без теста такой сбой заметен только глазами

Зачем CI

```
$ pytest  
===== 12 passed =====
```

```
$ pytest    # свежий клон репозитория  
===== 12 failed =====  
ERROR: file 'config.json' not found
```

Зачем CI

```
$ pytest  
===== 12 passed =====
```

```
$ pytest    # свежий клон репозитория  
===== 12 failed =====  
ERROR: file 'config.json' not found
```

Локально «всё работает», потому что забыли закоммитить нужный файл. CI запускает всё с нуля и **не даёт обманывать ни себя, ни других**

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Что такое форматтер и code style

- **Code style** — договорённости о том, **как выглядит** код: имена, отступы, скобки, длина строк
- **Форматтер** — программа, которая **автоматически** приводит код к единому виду, не меняя его смысла
- Форматтеров и стилей много, важно не «какой лучше», а чтобы в одном проекте был **один** стиль

Пример: PEP 8 — стандарт оформления для Python. Конкретные правила значения не имеют: главное, что сообщество договорилось об одном варианте

Общие принципы code style

- Имена: внятные, единый стиль для похожего
- Один и тот же отступ по всему коду
- Аккуратные пробелы вокруг операторов
- Разумная длина строк
- Пустые строки между смысловыми блоками
- Одно действие — одна строка

Правила существуют в любом языке, вопросы только в деталях: у Python (PEP 8) свои, у C (K&R, GNU) свои, у Rust (rustfmt) свои

Форматтеры в разных языках

Для многих языков есть де-факто стандартные форматтеры:

Язык	Форматтер
Python	ruff, black
C	clang-format
Kotlin	ktlint
Rust	rustfmt
Pascal	ptop, встроенный форматтер Lazarus IDE

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Статические и динамические

	Когда	Что находят	Примеры
Статические	до запуска кода	стиль, неиспользуемые переменные, сомнительные конструкции	линтеры, компиляторы, проверки типов
Динамические	во время выполнения	утечки, падения, реально выполненные строки	санитайзеры, профайлеры, coverage

Линтер — самый известный статический инструмент: проверяет код на стиль и типовые ошибки

Статические проверки дешёвы — но ограничены доступной информацией

Динамические дороги — но могут находить гораздо более сложные ошибки

Типы — это статический анализ

Со статической типизацией: ошибка видна ещё до запуска

```
int double_it(int x)    { return x * 2; }  
int y = double_it("hello"); // ошибка  
компиляции
```

Без строгих типов: та же ошибка всплывает только на конкретном запуске

```
def double_it(x):  
    return x * 2  
  
double_it("hello")  
# TypeError уже при исполнении
```

Статические анализаторы по языкам

Язык	Анализаторы
Python	ruff, mypy, pyright
C	gcc -Wall, clang-tidy, cppcheck
Kotlin	detekt, ktlint
Rust	cargo clippy (и сам borrow checker)
Pascal	предупреждения FPC, анализ в Lazarus IDE

Динамические анализаторы

- **Санитайзеры** (например, AddressSanitizer) — утечки памяти, выходы за границы массива
- **Coverage** — какие строки на самом деле выполнились под тестами
- **Профайлеры** — где программа реально тратит время и память

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Мотивация

- Любая программа содержит ошибки
- Если программа **не содержит** ошибок, их содержит алгоритм, который реализует эта программа
- Если ни программа, ни алгоритм ошибок не содержат, такая программа даром никому не нужна

Ошибки есть всегда, но надо минимизировать их количество и влияние

Почему? Потому что некоторые ошибки очень дорогие (а иногда вообще могут быть причиной смерти)

Хронология ошибок



Источник: *презентация Владимира Кутуева*

Виды тестов

- **Функциональные:** проверяют, что программа делает то, что должна
- **Нефункциональные:** проверяют иные важные свойства
 - производительность
 - безопасность
 - удобство использования
 - ...

Это, как и многое в программировании, лишь один из возможных способов смотреть на тесты

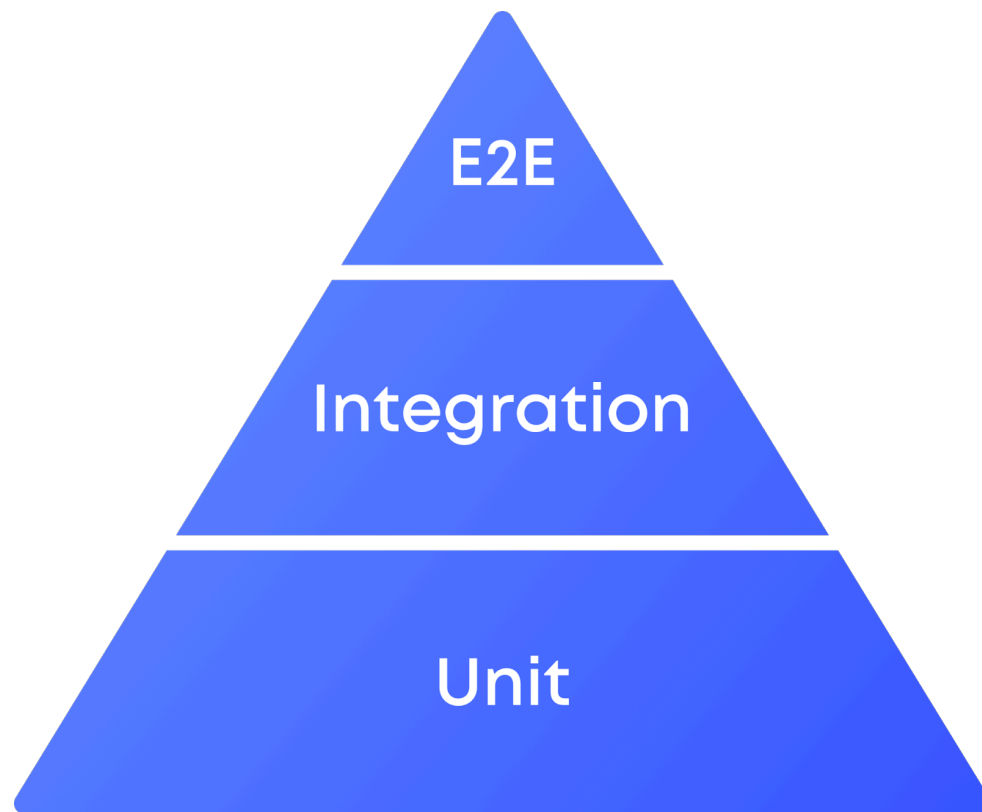
Уровни тестирования

Выше — сложнее и дороже запускать

Ниже — быстрее и проще

Unit	отдельные компоненты программы
Integration	взаимодействие компонентов между собой
End-to-end	система в целом, как её видит пользователь

Уровней можно придумать больше, границы подвижны: хороший набор тестов сочетает все уровни



Unit-тесты

- Проверяют **одну компоненту**
- Должны быть **быстрыми**
- Проверяют разные, особенно **краевые**, случаи
- Их должно быть **много**
- Запускаются при каждом изменении кода $\Rightarrow$ помогают при рефакторинге
- Являются отличной **документацией**
- Помогают улучшить **архитектуру**

Важно: unit-тесты **не доказывают** отсутствие ошибок в программе. Они лишь делают ошибки дешевле

Тестовые фреймворки

В каждом языке есть свои инструменты для тестов:

Язык	Тестовый фреймворк
Python	pytest, unittest
C	CTest, Check, GoogleTest
Kotlin	JUnit, kotest
Rust	встроенный cargo test

Идея одна и та же: описания тестов + запуск по команде + компактный отчёт о том, что упало

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Принцип TDD

Тесты пишутся **до** реализации:

1. Красный

Пишем тест нужного поведения. Он не может не упасть — реализации ещё нет

2. Зелёный

Пишем **минимум** кода, чтобы тест прошёл

3. Рефакторинг

Улучшаем код. Тест остаётся зелёным — ничего не сломали

Короткий цикл: **красный** ⇒ **зелёный** ⇒ **рефакторинг**, и так на каждом маленьком шаге

Зачем TDD

- Тест **до** кода — это формулировка задачи, а не способ проверить результат
- Заставляет думать о **поведении** снаружи, а не о реализации внутри
- Даёт принцип **fail-fast**: как только рефакторинг что-то сломал, тест падает сразу

Именно такие тесты проще всего передать другому человеку (или машине) как **контракт** поведения

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Всё в одном месте

Каждый инструмент — отдельная программа: `ruff`, `mypy`, `pytest`, ... Запускать их из терминала по очереди неудобно: медленно, легко забыть какой-то из них

Всё в одном месте

Каждый инструмент — отдельная программа: `ruff`, `mypy`, `pytest`, ... Запускать их из терминала по очереди неудобно: медленно, легко забыть какой-то из них

- **Подсветка проблем** прямо в коде: формат, типы, линт
- **Форматирование при сохранении** (format-on-save)
- **Запуск тестов и анализаторов** одной кнопкой
- Инструменты работают **в фоне**, не отвлекая от кода

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Что такое CI

- Автоматическая **сборка**, запуск **тестов**, **линтеров**, ...
- Запускается на каждом изменении: push, pull request
- Убеждаемся, что **актуальное состояние** проекта хотя бы собирается и проходит проверки
- Не даём обманывать **ни себя, ни других**

По сути — те же команды, что вы запускаете локально, но на **чистой машине** и без пропусков

CI — не только GitHub Actions

Система	Где встречается
GitHub Actions	GitHub
GitLab CI	GitLab
Jenkins, TeamCity, ...	свои серверы, корпоративные процессы

Пример: GitHub Actions

```
name: CI
on: [push, pull_request]
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: pip install ruff pytest
      - name: Format
        run: ruff format --check .
      - name: Lint
        run: ruff check .
      - name: Tests
        run: pytest
```

Такой YAML лежит в репозитории и запускается сам: на push и на pull request. Форматирование, линт и тесты выполняются **без участия человека**

CI — контракт команды

- Для каждого изменения CI прогоняет **форматирование, линт и тесты**
- Пока проверки красные — изменение **не попадает** в основную ветку
- Правило команды: **main всегда зелёный**

CI превращает правила из «мы договорились» в «это невозможно нарушить». Форматирование, анализаторы и тесты начинают работать **вместе**, как единый конвейер

Содержание

Мотивация	1
Зачем форматтер	2
Зачем анализаторы	3
Зачем тесты	4
Зачем CI	5
Форматирование и code style	6
Что такое форматтер и code style	7
Общие принципы code style	8
Форматтеры в разных языках	9
Анализаторы	10
Статические и динамические	11
Типы — это статический анализ	12
Статические анализаторы по языкам	13
Динамические анализаторы	14
Тесты: введение	15
Мотивация	16
Хронология ошибок	17
Виды тестов	18
Уровни тестирования	19
Unit-тесты	20
Тестовые фреймворки	21
TDD (Test-Driven Development)	22

Принцип TDD	23
Зачем TDD	24
Инструменты и IDE	25
Всё в одном месте	26
Continuous Integration (CI)	27
Что такое CI	28
CI — не только GitHub Actions	29
Пример: GitHub Actions	30
CI — контракт команды	31
В эпоху ИИ	32
Зачем инструменты, если код пишет ИИ	33
Почему TDD снова актуален	34
Резюме	35

Зачем инструменты, если код пишет ИИ

- Нейросети выдают очень много кода, но **не гарантируют** его правильность
- Проверять сгенерированный код глазами нереально — нужен **оракул**: тесты
- Форматтеры и анализаторы приводят бесконечный поток вариантов к **единому стилю** и ловят подозрительные места
- CI берёт на себя проверку этого потока **при каждом изменении**

Чем больше кода производится, тем важнее **машинная** верификация

Почему TDD снова актуален

- Тест — это **спецификация**, которую можно буквально скормить модели
- «Приёмочные тесты» становятся постановкой задачи: ИИ пишет реализацию, тесты решают, верно ли
- Возвращаемся к TDD: сначала **контракт** (тест), потом реализация — руками или с помощью ИИ

Вопрос «поняла ли модель задачу» теперь имеет проверяемый ответ: **запустите тесты.**

Резюме

- Тулинг — это **дисциплина**, а не бюрократия
- **Форматтер**: единый стиль и чистые диффы
- **Анализаторы**: ловят ошибки до прода
- **Тесты**: уверенность в изменениях
- **CI**: общие правила, которые невозможно нарушить

Все вместе они делают программу не только **написанной**, но и **поддерживаемой** — а в эпоху ИИ этого не отменить